```
  return
true
```

#1 Instead of handling removal from the tail of a linked list, we just remove the top element from a priority queue. Invariant: if the hash table is not empty, the queue must not be empty

7.5.3 Performance

Let’s once again update our table with the performance for LFU. All writing operations now involve modifying a heap, so we can’t have the constant time guarantee anymore, except for the lookup.

It’s worth noting that with a *Fibonaccy Heap*, instead, insertion and update of priority in the heap would run in amortized constant time, but deleting the top element would still require logarithmic time, and so we couldn’t improve the asymptotic running time needed for storing an entry (since it in turn uses delete).

Table 7.3 Comparative analysis of performance vs implementation for cache with n elements

	Array	List	Hash Table	Tree	LRU Cache	LFU Cache
Storing an entry	O(1)	O(n)	O(1) #	O(log n)	O(1) #	O(log n) #